

ft_newton

Some Models to describe the world

Summary: In this project, you will code a basic physics engine (like PhysX, Havok, or Bullet Physics Engine)

Version: 2.2

Contents

I	**Preamble**	**2**
II	**AI Instructions**	**3**
III	**Introduction**	**5**
IV	**Instructions**	**6**
V	**Mandatory part**	**7**
VI	**Bonus Part**	**9**
VII	**Submission and peer-evaluation**	**10**

Chapter I

Preamble

Truth is ever to be found in simplicity, and not in the multiplicity and confusion of things.
— Sir Isaac Newton

Chapter II

AI Instructions

🟢 Context

AI is now a powerful coding partner — alongside your peers — for tackling large and demanding projects. You will guide it through both technical and non-technical aspects of your work.

AI tools can boost your efficiency and improve the quality of your output, but you should be able to dive deep into any part of the project without relying on them.

Your AI partner supports you, but you remain fully responsible for making informed technical decisions and to clearly explain and defend them.

🟢 Main message

- 👉 Strive for a mature and responsible use of AI.
- 👉 Never let AI take responsibility for decisions — especially when it lacks awareness of your goals, constraints, or team dynamics.
- 👉 Maintain creativity, innovation, and human oversight through active collaboration with your peers. AI is trained on existing data and rarely generates truly new ideas.
- 👉 Stay informed about emerging trends and be ready to adapt to new concepts and technologies.

🟢 Learner rules:

- Maintain intellectual leadership over your projects and make your own informed decisions.
- Prioritise the collective intelligence of your team and peers.
- Actively stay informed about the ongoing evolution of AI technologies.

Phase outcomes:

- AI engineering skills.
- Increased efficiency.
- Greater reliability and quality.
- A pioneering mindset.

Comments and examples:

- Your peers can identify trade-offs, question assumptions, and help you improve. The first answer from an AI might not be the best — it may lack efficiency, security, or real added value. Now more than ever, you should rely on your peers.
- AI can make you faster, but your peers make you better. Collaboration, discussion, and mutual challenge are key to success.
- Be transparent about how AI was used in your projects, and clearly identify what was generated by AI tools.

✓ Good practice:

> I asked AI to help generate unit tests for my API. I reviewed them with my teammate, and we adjusted them for edge cases. It saved time, and we both learned something new.

✗ Bad practice:

> I had AI generate the entire architecture of my project. It "works," but when I'm asked to explain design decisions during the peer review or in front from of a customer, I cannot. I lose credibility and I fail.

Chapter III

Introduction

This project introduces the physic engine, a part of a classical game engine.
You will learn about collision detection, rigid body, elastic collisions, Torque, inertia tensor, ...
Go as far as you want regarding maths and physics but don't take too much time.
Sometimes the best is the enemy of the good.
You'll be able to make a better version of your code in your next project.

Chapter IV

Instructions

General instructions :

- You must use a GPU API for 3D rendering (like Vulkan, Metal, OpenGL, WebGPU, ...), with shaders and render pipeline.
- You can use any 3D Asset, any audio file, free of copyright. The artistic quality is not part of the correction scale.
- You must code the object's interaction, rotation, movement, and collision yourself. No external library for calculation is allowed.
- Code in C, C++, or Rust.
- Nothing should do physics or collision-related stuff for you, be smart!
- Do you need exception handling, RTTI and such?
- A simple 'make' command should be enough to compile the full project.

Chapter V

Mandatory part

You will code a small Angry Birds inspired game:

- A scene where you use either a Catapult or a Trebuchet (this choice is a game-changer) throwing some apples (projectiles) on some unstable composed structures.
- Impact must produce a series of reaction that is physically correct and the system must go back to a stable state (no infinite bounce).
- You will code at least 3 primitive colliders based on different geometries: a box, a sphere, and a plane.
- The game play can be 2D only (like the real angry bird), but your physic and rendering engine must be in 3D.
- It must be easy to spawn a LOT of physical objects on demand.
- An FPS counter and an object counter must be present.
- The scene should be playable with a normal user display without any debugging information.
- A full and direct control of at least: time (reversal is not mandatory), gravity, projectile speed, projectile direction, projectile mass.
- A debug display, that prints the colliders as wireframes, is mandatory and can be turned on/off at any time.
- Everything must be considered as a rigid body (no deformation).
- You will implement the translational motion.
- You will implement rotational motion (it's harder, but you don't need a perfect version here).
- You will implement elastic collisions.
- You will implement gravity.
- You must be able to show every feature in the main game easily (no recompile or 5 minutes setup).

You are free to do anything compatible with those rules.



You need a working simulation that can handle a lot of objects (no brute force!) with basic behaviour. Advance physics and simulation are welcome but not mandatory.

Chapter VI

Bonus Part

- Big: good ragdoll.
- Medium: A mesh collider that can create an accurate collider based on 'any' 3D models. You need to prepare at least 3 models including 1 humanoid and be prepared to show the precision of the collider with specific testing.
- Small: anything, a nice GUI, better primitive, advanced physics.

Here again, keep big ideas for Very Real Engine.



The bonus part will only be assessed if the mandatory part is PERFECT. Perfect means the mandatory part has been integrally done and works without malfunctioning. If you have not passed ALL the mandatory requirements, your bonus part will not be evaluated at all.

Chapter VII

Submission and peer-evaluation

Submit your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defence. Don't hesitate to double check the names of your folders and files to ensure they are correct.

If your assets are over 1Go replace your asset folder with a clean md5 recursive hash of the folder and bring this folder to your correction.